

AKRAM MOHAMED

West Lafayette, IN 47906

(765) 491-1823 ◊ akramreda120@gmail.com ◊ linkedin.com/in/akram-r-mohamed ◊ github.com/Akram4212

EDUCATION

Purdue University

West Lafayette, IN

Bachelor of Science in Electrical Engineering

May 2026

Relevant Coursework: Computer Architecture, Digital Integrated Circuit Analysis and Design, ASIC Design, VLSI Design, Microprocessor Systems and Interfaces, Data Structures, Advanced C Programming, Python for Data Science

SKILLS

RTL Design & Digital Logic: Verilog, SystemVerilog, synthesizable RTL, FSMs, datapath/control logic, pipelining, synchronous design, reset strategy awareness, digital IP design

CPU & SoC Architecture: RISC-V, RV32I, CPU microarchitecture, instruction decode, branch handling, hazard detection, forwarding, stalls, flushes, load/store behavior, memory-mapped peripherals

Caches & Memory Systems: Private I-cache/D-cache design, cache hit/miss behavior, valid/dirty metadata, write-back behavior, bus snooping, invalidation logic, modified-line tracking, LR/SC atomic operations, AXI/AHB-style interfaces

Verification & Debug: SystemVerilog testbenches, cocotb/Python testbenches, directed/randomized testing, SVA concepts, protocol assertions, functional coverage tracking, regression testing, waveform debug, root-cause analysis

ASIC/FPGA Flow: RTL simulation, logic synthesis, FPGA implementation, timing/resource analysis, timing constraint awareness, PPA tradeoff analysis, CDC exposure, DFT exposure, physical design awareness

Programming & Tools: Python, C, C++, embedded C, Tcl, Linux/Unix shell, Git, Make/CMake, GitHub Actions, DPI-C, Synopsys VCS, QuestaSim/ModelSim, Verilator, Vivado, GTKWave, Cadence Virtuoso

PROJECT EXPERIENCE

AXI-Lite RISC-V SoC and RV32I Core Verification Platform — *SystemVerilog, cocotb, Python, Verilator, GTKWave* June 2026 – Present

- Designed a five-stage RV32I-style processor core in SystemVerilog with PC control, decode, register file, ALU, pipeline latches, forwarding, hazard handling, branch/JALR redirect logic, load/store handling, and writeback control.
- Integrated the core into an AXI-Lite based SoC platform with memory-mapped GPIO, timer, RAM, AXI-Lite interconnect, protocol assertions, and subsystem-level regression tests.
- Built cocotb/Python verification for RV32I execution, ALU operations, decoder behavior, register file behavior, forwarding priority, load-use hazards, branches, jumps, memory operations, and instruction/data stalls.
- Developed reusable AXI-Lite master utilities, directed/randomized tests, functional coverage tracking, byte-strobe checks, invalid-address tests, and CI regression workflows using Verilator and GitHub Actions.

Multicore RISC-V CPU RTL Design with Cache Coherence and Atomic Operations — *SystemVerilog, Vivado, VCS, QuestaSim* Jan – May 2026

- Designed synthesizable SystemVerilog RTL for a multicore RISC-V CPU system, implementing pipelined execution, instruction decode, branch handling, hazard logic, forwarding, stalls, flushes, and memory-stage control.
- Designed a multicore cache subsystem with private instruction/data caches, tag/index comparison, valid/dirty metadata, hit/miss detection, write-back behavior, bus-based snooping, invalidation, and modified-line tracking.
- Implemented LR/SC atomic transaction support for reservation tracking, store-conditional behavior, and synchronization across core/cache memory operations.
- Built directed tests and regression scenarios for ALU operations, branch behavior, load-use hazards, forwarding paths, flush logic, cache coherency behavior, atomic operations, and memory transactions.
- Synthesized and characterized the design up to 77.68 MHz with 7,682 slice LUTs, supporting timing, resource, and PPA-style analysis.

Memory-System Performance Modeling and Validation — *C++, SystemVerilog, DPI-C, Python* Jan – May 2026

- Integrated the C++ Ramulator2 DRAM simulator with a SystemVerilog environment through DPI-C to validate memory timing, response ordering, outstanding requests, and backpressure behavior.
- Modeled AXI-style read/write transactions, burst transfers, reorder buffering, expected responses, and DRAM timing behavior for processor and accelerator memory validation.
- Implemented shadow-memory checking, transaction tracing, expected-vs-actual validation, and debug logs to isolate memory correctness, ordering, latency, and throughput issues.
- Automated build, simulation, regression, log collection, and performance-analysis workflows using C++, Python, Tcl, Make/CMake, Linux shell, and Git.

USB Full-Speed Digital IP RTL Design and Functional Verification — *SystemVerilog, AHB, USB* Dec 2025

- Designed a USB Full-Speed SoC peripheral with an AMBA AHB subordinate interface for memory-mapped processor communication.
- Implemented RTL datapath and control logic for USB RX/TX behavior, packet decoding, transfer state machines, protocol error detection, buffering, and flow control.
- Built SystemVerilog testbenches with protocol checks, scoreboard-style validation, and directed tests for USB packet handling, AHB transactions, interface behavior, and error cases.
- Debugged simulation failures using waveform analysis, transaction traces, simulation logs, and directed tests to validate subsystem-level functionality.